Course Discussions

# Solution: Remove Duplicate Letters

## Problem Statement

Given a string `s`, remove all duplicate letters from the input string while maintaining the original order of the letters.

Additionally, the returned string should be the smallest in lexicographical order among all possible results.

**Examples**:

1. 
   - **Input**: "bbaac"
   - **Expected Output**: "bac"
   - **Justification**: Removing the extra 'b' and one 'a' from the original string gives 'bac', which is the smallest lexicographical string without duplicate letters.
2. 
   - **Input**: "zabccde"
   - **Expected Output**: "zabcde"
   - **Justification**: Removing one of the 'c's forms 'zabcde', the smallest string in lexicographical order without duplicates.
3. 
   - **Input**: "mnopmn"
   - **Expected Output**: "mnop"
   - **Justification**: Removing the second 'm' and 'n' gives 'mnop', which is the smallest possible string without duplicate characters.

## Solution

To solve the given problem, begin by iterating through the string, and calculate the frequency count for each character. We will use the stack to maintain the order of characters and the set to track the uniqueness of characters.

Next, start traversing the string, and for every character encountered, check if it's already in the 'present' set. If it's not, compare it with the top element of the 'result' stack. If the stack is not empty, and the current character is lexicographically smaller than the stack's top, and the top character of the stack appears again in the string (indicated by a non-zero frequency count), repeatedly pop the stack and remove those elements from the 'present' set. Then, add the current character to the stack and the 'present' set.

This process, facilitated by the stack and set, ensures that the stack is built with unique characters in the smallest lexicographical order. After processing the entire string, pop elements from the stack to construct the final string, thereby obtaining the smallest lexicographical sequence without duplicate characters.

1. **Frequency Count** (`count`):
   - Initialize a `count` dictionary (or hash map in some languages) to store the frequency of each character in the string `s`.
2. **Character Presence Tracking** (`present`):
   - Use a set `present` to keep track of the characters that have been added to the resultant string. This set prevents duplicate characters in the result.
3. **Building the Result** (`result`):
   - Create a stack `result` to construct the final string.
   - For each character `c` in the string `s`:
     - If `c` is not in `present`, proceed to compare it with the top character of `result`.
     - While `result` is not empty, and `c` is lexicographically smaller than the top character of `result`, and the frequency of the top character of `result` is more than 0 (indicating it appears again in the string), pop the top character from `result` and remove it from `present`.
     - Push `c` onto `result` and add it to `present`.
     - Decrement the frequency of `c` in `count`.
4. **Result Construction**:
   - The stack `result` now contains the characters of the final string in reverse order. Pop elements from `result` to construct the output string in the correct order, from left to right.

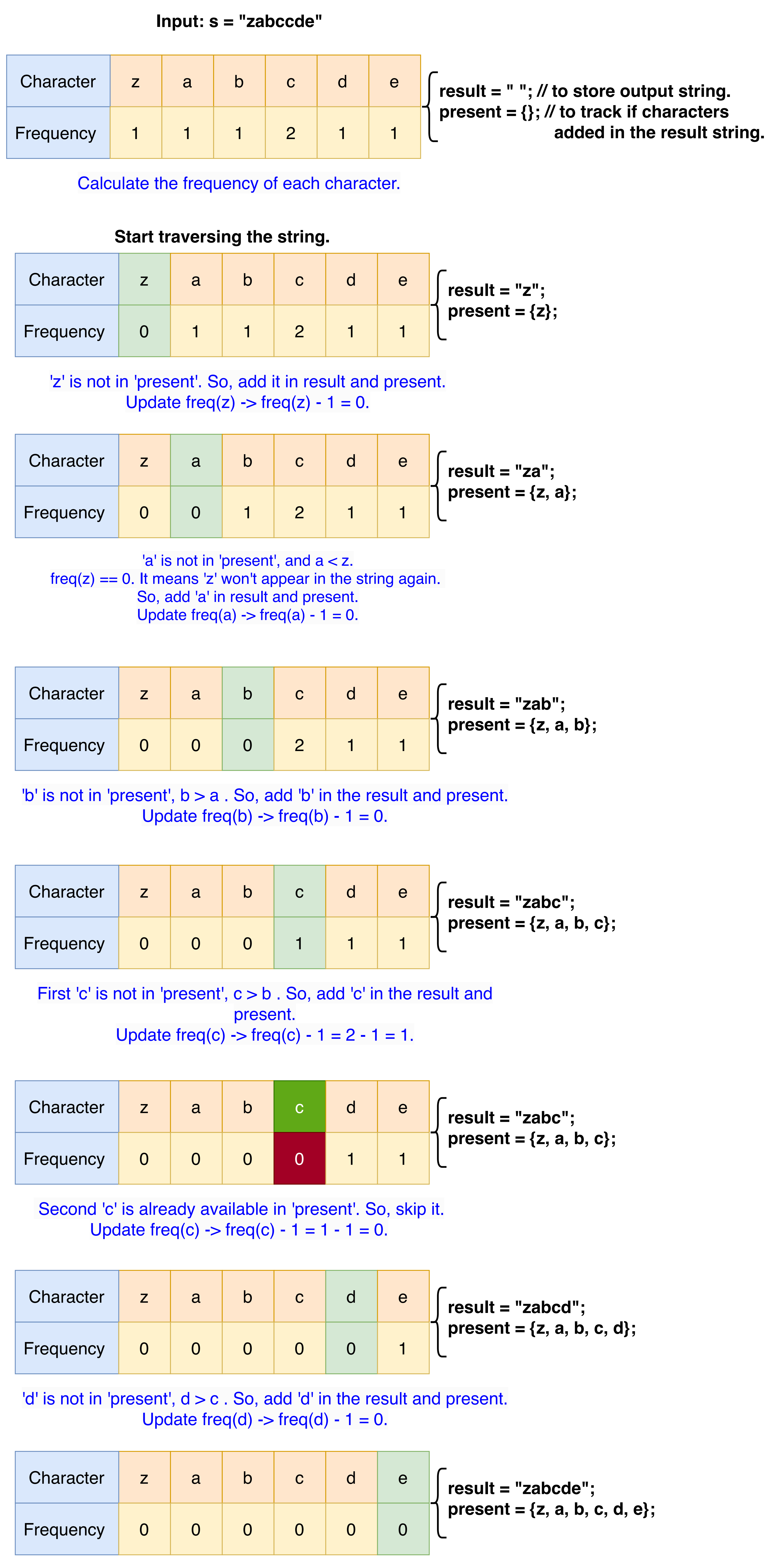
This approach works because it ensures that the smallest character is placed first, respecting the original order and removing duplicates. The frequency count ensures that characters are not wrongly discarded.

**Algorithm Walkthrough**:

**Input**: "zabccde"

1. **Initialization**:
   - Calculate the frequency of each character: `{'z': 1, 'a': 1, 'b': 1, 'c': 2, 'd': 1, 'e': 1}`.
   - Initialize an empty stack `result` and a set `present` to keep track of characters already in `result`.
2. **Iteration Over Characters**:
   - '**z**': Not in `present`. Add 'z' to `result`, add to `present`. Decrease frequency of 'z'.
   - '**a**': Not in `present`. As 'a' is smaller than 'z', but 'z' won't appear again (frequency is now 0), we keep 'z'. Add 'a' to `result`, add to `present`. Decrease frequency of 'a'.
   - '**b**': Not in `present`. Add 'b' to `result`, add to `present`. Decrease frequency of 'b'.
   - **First** '**c**': Not in `present`. Add 'c' to `result`, add to `present`. Decrease frequency of 'c'.
   - **Second** '**c**': Already in `present`. Skip it.
   - '**d**': Not in `present`. Add 'd' to `result`, add to `present`. Decrease frequency of 'd'.
   - '**e**': Not in `present`. Add 'e' to `result`, add to `present`. Decrease frequency of 'e'.
3. **Result Construction**:
   - The `result` stack now contains `['z', 'a', 'b', 'c', 'd', 'e']`.
   - Convert the stack to a string to get the final result: "zabcdef".

**Expected Output**: "zabcdef"



**Input: s = "zabccde"**

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 1 | 1 | 1 | 2 | 1 | 1 |

result = " "; // to store output string.
present = {}; // to track if characters added in the result string.

Calculate the frequency of each character.

**Start traversing the string.**

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 1 | 1 | 2 | 1 | 1 |

result = "z";
present = {z};

'z' is not in 'present'. So, add it in result and present.
Update freq(z) -> freq(z) - 1 = 0.

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 1 | 2 | 1 | 1 |

result = "za";
present = {z, a};

'a' is not in 'present', and a < z.
freq(z) == 0. It means 'z' won't appear in the string again.
So, add 'a' in result and present.
Update freq(a) -> freq(a) - 1 = 0.

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 0 | 2 | 1 | 1 |

result = "zab";
present = {z, a, b};

'b' is not in 'present', b > a . So, add 'b' in the result and present.
Update freq(b) -> freq(b) - 1 = 0.

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 0 | 1 | 1 | 1 |

result = "zabc";
present = {z, a, b, c};

First 'c' is not in 'present', c > b . So, add 'c' in the result and present.
Update freq(c) -> freq(c) - 1 = 2 - 1 = 1.

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 0 | 0 | 1 | 1 |

result = "zabc";
present = {z, a, b, c};

Second 'c' is already available in 'present'. So, skip it.
Update freq(c) -> freq(c) - 1 = 1 - 1 = 0.

| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 0 | 0 | 0 | 1 |

result = "zabcd";
present = {z, a, b, c, d};

'd' is not in 'present', d > c . So, add 'd' in the result and present.
Update freq(d) -> freq(d) - 1 = 0.

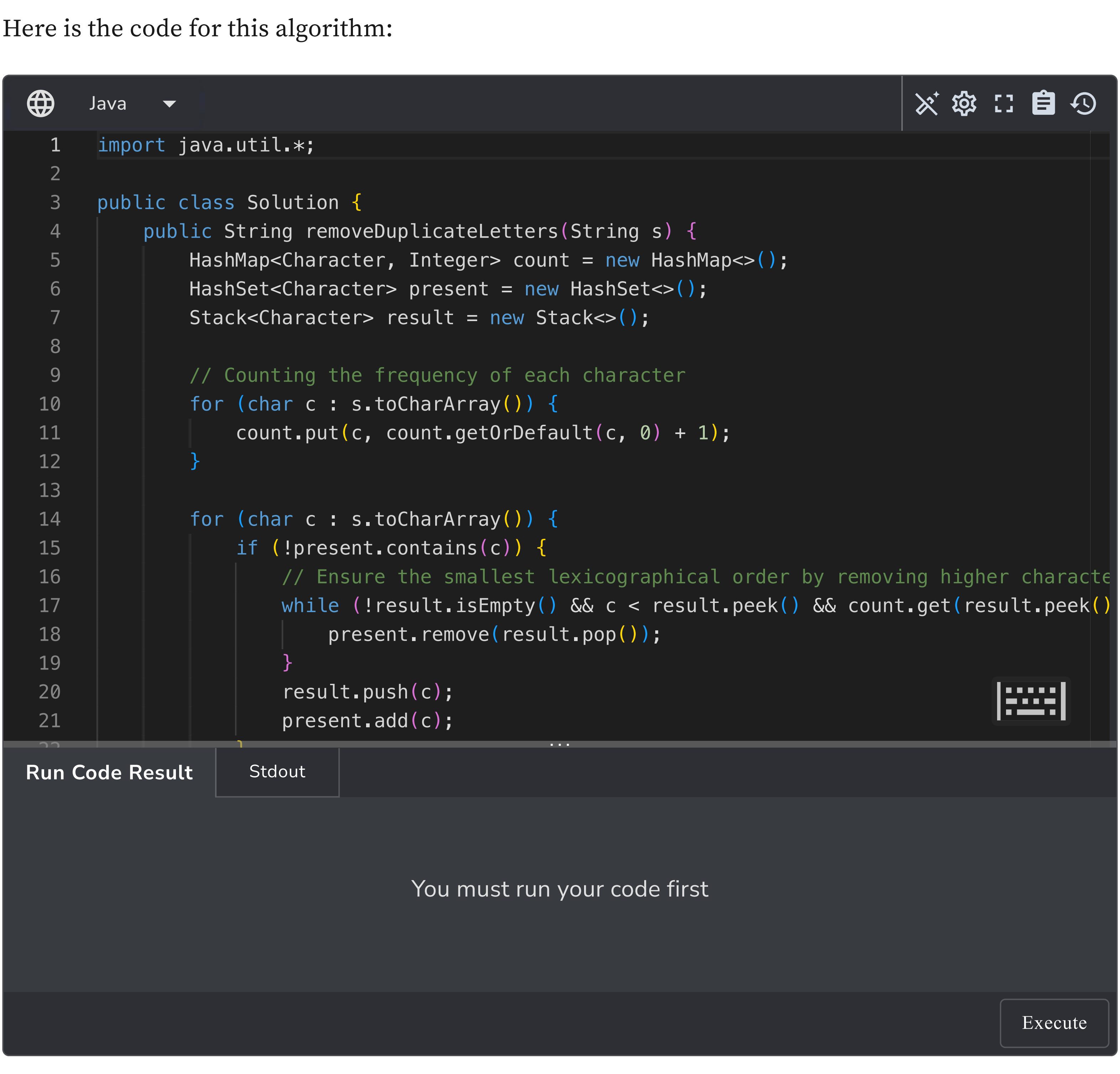
| Character | z | a | b | c | d | e |
|---|---|---|---|---|---|---|
| Frequency | 0 | 0 | 0 | 0 | 0 | 0 |

result = "zabcde";
present = {z, a, b, c, d, e};

'e' is not in 'present', e > d . So, add 'e' in the result and present.
Update freq(e) -> freq(e) - 1 = 0.

## Code

Here is the code for this algorithm:



```java
import java.util.*;

public class Solution {
    public String removeDuplicateLetters(String s) {
        HashMap<Character, Integer> count = new HashMap<>();
        HashSet<Character> present = new HashSet<>();
        Stack<Character> result = new Stack<>();

        // Counting the frequency of each character
        for (char c : s.toCharArray()) {
            count.put(c, count.getOrDefault(c, 0) + 1);
        }

        for (char c : s.toCharArray()) {
            if (!present.contains(c)) {
                // Ensure the smallest lexicographical order by removing higher characte
                while (!result.isEmpty() && c < result.peek() && count.get(result.peek()
                    present.remove(result.pop());
                }
                result.push(c);
                present.add(c);
```

You must run your code first

## Complexity Analysis:

- **Time Complexity**: O(N) where N is the length of the string. Each character is visited once.
- **Space Complexity**: O(1) as the extra space used does not depend on the input size. The character count and the stack size are bounded by the character set size (constant).

← Previous
Remove Duplicate Letters (Medium)

Next →
Next Permutation (Medium)

☐ Mark as Completed